

16782 HW1

Rashed Albalawi

9/28/2026

Authorship and AI-Assistance Statement

All project-specific ideas, and implementation were developed and written by the author. *Artificial Intelligence: A Modern Approach*, fourth edition, was used extensively as a general reference for artificial-intelligence concepts and search methods [1].

AI-Assistance Statement

Generative AI tools were used in a supporting role for debugging, LaTeX assistance, limited code modifications, and language refinement. They were not used to originate ideas.

Contents

1 Problem	3
1.1 Goal	3
1.2 Environment	3
2 Methodology	3
2.1 Problem Formulation	3
2.2 Planning Algorithms	4
2.3 Heuristic	5
2.3.1 Earliest Interception	5
2.3.2 Departure Feasibility	5
2.3.3 Spatial Cost	6
2.3.4 Heuristic Evaluation	7
3 Implementation Details	7
3.1 Data Structures	7
3.2 Search Algorithms	8
3.3 Departure Feasibility and Spatial-Cost Learning	8
3.4 Planning Algorithm	9
3.4.1 Heuristic Refinement	9
3.4.2 Local Heuristic Updates	10
4 Completeness & Optimality	11
5 Results	12
References	34

1 Problem

1.1 Goal

A target moves one step at a time in an 8-connected grid environment. Its trajectory is known to the robot prior to the start of the planning process. The goal is to catch the target while accumulating as little cost as possible.

1.2 Environment

The environment is an 8-connected grid deterministic and fully observable. The robot knows the costs and occupancy of all cells in the environment at all times including the target's trajectory.

The environment can be as large as 2000×2000 cells, while the target trajectory can span around 5500 time steps. A full A* search over this space is infeasible because its worst-case complexity is $O(b^d)$, where b is the branching factor and d is the search depth. Here, $b = 9$, since the robot may move to any neighboring cell or remain still, and d can reach 5500. Therefore, A* may expand up to 9^{5500} nodes in the worst case, which is computationally impractical.

On top of this computational challenge the robot must plan in real-time as the target is moving, so the planner must be able to make decisions quickly and efficiently.

2 Methodology

2.1 Problem Formulation

The environment is represented as an 8-connected grid graph with valid cells V . A robot state is $s = (p, t)$, where $p = (x, y) \in V$ is its position and t is the current time step. Each action moves the robot to a neighboring cell or keeps it in place, advances time by one step, and has a traversal cost $c(p, p')$.

The known target trajectory is $Q = (q_0, q_1, \dots, q_T)$, where $q_\tau = (x_\tau, y_\tau)$ is the target position at time τ and T is the final time step. An interception occurs when the robot reaches q_τ at the same time τ . The objective is to find a valid interception path $\pi = (p_t, \dots, p_\tau)$ that minimizes

$$J(\pi) = \sum_{k=t}^{\tau-1} c(p_k, p_{k+1}), \quad p_\tau = q_\tau.$$

For convenience, let $\mathcal{Q} = \{q_\tau \mid 0 \leq \tau \leq T\}$ denote the set of all cells visited by the target. Let $\mathcal{Q}_f$ denote the subset containing the feasible target cells.

2.2 Planning Algorithms

In this problem, the target moves in real time, and the robot must intercept it before the target reaches the end of its trajectory. In principle, a full A* or Dijkstra search could solve the complete 3D planning problem. In practice, however, such a search requires significant computation and might not finish within the available planning time. The size of the environment and the length of the target trajectory make this approach impractical here.

Real-time heuristic search addresses this limitation by interleaving planning and execution. Some real-time algorithms, such as those considered in this work, operate as follows: at each time step, the planner performs a bounded A* search from the robot's current state, selects the next action, and then uses the local search results to improve the heuristic for future searches rather than discarding all of the information.

In this work, we consider the following real-time heuristic search algorithms:

- Local Search Space LRTA (LSS-LRTA) [2][3].
- Real-Time Adaptive A (RTAA) [4].

Let C be the set of closed states by the bounded A* search, let O be the set of open states in that local search space but not closed. The algorithms differ primarily in their learning step:

- **LSS-LRTA.** LSS-LRTA uses a dynamic-programming update over the local search space. In this implementation, a backward Dijkstra search is used. For every expanded state $s \in C$, the update is

$$h_{\text{new}}(s) = \min_{u \in O} \{d_C(s, u) + h(u)\},$$

This update produces more informed heuristic values, but requires a backward shortest-path computation over the local search space.

- **RTAA.** RTAA first computes the minimum open f -value

$$f_{\min} = \min_{u \in O} \{g(u) + h(u)\}.$$

It then updates every expanded state using

$$h_{\text{new}}(s) = f_{\min} - g(s), \quad s \in C.$$

This update requires only a linear pass over the expanded states. It is therefore faster than the LSS-LRTA learning step, although its heuristic updates can be less informed.

Both algorithms use the same bounded-lookahead planning framework. They differ only in the local heuristic update applied after the lookahead search. The complete framework and the two update procedures are presented in Section 3.4.

2.3 Heuristic

The heuristic combines three components:

1. **Earliest interception:** Estimates the earliest possible interception using Chebyshev distance.
2. **Departure feasibility:** Checks whether interception is still possible before the trajectory ends.
3. **Spatial cost:** Estimates the minimum cost to the nearest target position while accounting for obstacles and traversal costs.

2.3.1 Earliest Interception

The earliest-interception heuristic gives the exact number of time steps needed to reach the target in a relaxed 3D space with no obstacles and a uniform traversal cost of 1. The Chebyshev distance to a target position is

$$d_{\infty}((x, y), (x_{\tau}, y_{\tau})) = \max(|x - x_{\tau}|, |y - y_{\tau}|).$$

The heuristic is then

$$h_{EI}(s) = \min_{t \leq \tau \leq T} \{\tau - t \mid \tau - t \geq d_{\infty}((x, y), (x_{\tau}, y_{\tau}))\}.$$

If no time step satisfies the condition, the heuristic returns ∞ .

At the initial state s_0 , the earliest-interception heuristic identifies the first target position that is feasible in the relaxed problem. Because feasibility remains true for all later target positions, the corresponding interception time sets the lower time bound for the states in $\mathcal{Q}_f$.

Since this is the exact cost-to-go for the relaxed problem, the heuristic is consistent and provides a lower bound on the cost of the original problem.

A direct approach checks each future target position until the interception condition is met, taking $O(N)$ time in the worst case for N remaining positions. However, since the target moves by at most one cell per time step, once the condition becomes true, it remains true for all later time steps. Therefore, binary search can find the earliest feasible interception time in $O(\log N)$.

2.3.2 Departure Feasibility

This is a state feasibility check rather than a cost-to-go estimate. It determines whether interception is still possible from a given state.

Let $d(p, q_\tau)$ be the minimum number of valid moves from p to q_τ while accounting for obstacles. Since each move takes one time step and the robot can wait, interception at time τ is possible when

$$t \leq \tau - d(p, q_\tau).$$

The latest feasible departure time from p is therefore

$$L(p) = \max_{q_\tau \in \mathcal{Q}_f} \{\tau - d(p, q_\tau)\},$$

A state is infeasible when $t > L(p)$.

This objective can be rewritten as a minimization by defining $D(p) = T - L(p)$:

$$D(p) = T - L(p) = \min_{q_\tau \in \mathcal{Q}_f} \{(T - \tau) + d(p, q_\tau)\}.$$

This can be computed using a backward multi-source Dijkstra search over the 2D grid. Each target position $q_\tau \in \mathcal{Q}_f$ is initialized with $T - \tau$, and each reverse move costs 1. Using the Dijkstra cost g , $D(p)$ is evaluated during the search as

$$D(p) = \begin{cases} g(p), & p \in \text{CLOSED}, \\ \min_{u \in \text{OPEN}} g(u), & p \notin \text{CLOSED}, \text{OPEN} \neq \emptyset, \\ \infty, & p \notin \text{CLOSED}, \text{OPEN} = \emptyset. \end{cases}$$

The feasibility value is then

$$h_{\text{DF}}(s) = \begin{cases} \infty, & D(p) > T - t, \\ 0, & \text{otherwise.} \end{cases}$$

2.3.3 Spatial Cost

The spatial-cost heuristic gives the exact cost of reaching the lowest-cost feasible target cell while accounting for obstacles and traversal costs in the relaxed 2D space (x, y) . For position p , the heuristic is

$$h_{\text{SC}}(p) = \min_{\substack{\pi=(p_0, \dots, p_k) \\ p_0=p, p_k \in \mathcal{Q}_f}} \sum_{i=0}^{k-1} c(p_i, p_{i+1}).$$

Since the time dimension is ignored, this value is a consistent lower bound on the actual cost in the full 3D space. It is computed using a backward multi-source Dijkstra search initialized with every cell in $\mathcal{Q}_f$. Using the Dijkstra cost g , the heuristic is then defined as

$$h_{SC}(p) = \begin{cases} g(p), & p \in \text{CLOSED}, \\ \min_{u \in \text{OPEN}} g(u), & p \notin \text{CLOSED}, \text{ OPEN} \neq \emptyset, \\ \infty, & p \notin \text{CLOSED}, \text{ OPEN} = \emptyset. \end{cases}$$

At any fixed stage of the search, this definition is consistent.

2.3.4 Heuristic Evaluation

The overall heuristic used is a the maximum of the three consistent heuristics defined above as follows:

$$h(s) = \max\{h_{DF}(s), h_{SC}(p), h_{EI}(s)\}.$$

This overall heuristic remains consistent as it is the maximum of consistent heuristics.

3 Implementation Details

3.1 Data Structures

The A* and Dijkstra search algorithms are implemented using the following data structures:

- **OPEN:**

- OPEN is implemented as a min-heap.
- Insertion and removal have $O(\log n)$ complexity.
- Accessing the minimum element (the top element) has $O(1)$ complexity.
- The C++ Standard Library `std::priority_queue` implementation is used.

- **CLOSED:**

- CLOSED is stored as a hash table.
- Insertion, deletion, and membership checking have $O(1)_a$ complexity.
- The C++ Standard Library `std::unordered_set` implementation is used.

- **Parent Pointers:**

- Implemented as a hash table.
- If a lower-cost parent to a state is found, it replaces the existing parent.
- Lookup, insertion, and update operations have $O(1)_a$ complexity.
- The C++ Standard Library `std::unordered_map` implementation is used.

- **Costs (g):**

- Implemented as a hash table.
- If a lower cost is found for a state, it replaces the existing value.
- Lookup, insertion, and update operations have $O(1)_a$ complexity.
- The C++ Standard Library `std::unordered_map` implementation is used.

- **States:**

- A 3D struct is used to represent the state, containing the x, y, and t coordinates.
- A custom hash function is defined for the struct so it can be used efficiently with hash tables.

3.2 Search Algorithms

The A* and Dijkstra search algorithms use the data structures described above. Duplicate entries are allowed in OPEN to avoid searching for an existing entry and updating its priority whenever a lower-cost path to the same state is found. The priority is f for A* and g for Dijkstra. Although OPEN may contain duplicate entries, CLOSED ensures that each state is expanded at most once; subsequent duplicates are discarded. For A*, this behavior requires a consistent heuristic, rather than one that is merely admissible.

An optimal path can be constructed by traversing the parent pointers from any expanded state to a start state.

3.3 Departure Feasibility and Spatial-Cost Learning

The learning process consists of two backward Dijkstra searches: one for departure feasibility and another for spatial cost. Both searches are initialized with Q_f , which is computed once during initialization using a single evaluation of the earliest-interception heuristic from the initial state.

- **Departure feasibility learning:**

- Departure feasibility is learned first because knowing which states are infeasible is more useful than improving their spatial-cost estimates.
- Spatial-cost learning starts only after departure-feasibility learning has finished.
- Once departure feasibility is fully learned, the planner can avoid moves from which the target can no longer be reached. This makes the algorithm complete with respect to finding a feasible interception.

- A possible source of incompleteness is the robot taking actions before departure feasibility has been fully learned.
 - When the minimum cost in OPEN exceeds the remaining time $T - t$, all states remaining in OPEN, as well as the unexplored states, are infeasible. Departure-feasibility learning can then stop, since those states will also remain infeasible at later time steps.
- **Spatial-cost learning:**
 - Spatial-cost learning begins after departure-feasibility learning has been completed.
 - It improves the cost estimates for reaching the target positions while accounting for obstacles and traversal costs.
 - The spatial-cost search continues incrementally whenever planning time remains.

3.4 Planning Algorithm

The two real-time planning algorithms use the same planning framework. They differ in their local update procedure $\mathcal{U}$: RTAA supplies RTAA-Update, while LSS-LRTA supplies LSS-LRTA-Update.

Algorithm 1: Real-time heuristic search planning

Input: current state s_{current} , heuristic H , time limit Δ , expansion bound N , local update procedure $\mathcal{U}$

```

1 if IsGoal( $s_{\text{current}}$ ) then
2   return;
3  $D \leftarrow \text{CurrentTime}() + \Delta$ ;
4  $D_{\text{learn}} \leftarrow \text{CurrentTime}() + 0.75 \cdot \text{RemainingTime}(D)$ ;
5 RefineHeuristic( $s_{\text{current}}$ ,  $H$ ,  $D_{\text{learn}}$ );
6  $R \leftarrow \text{BoundedAStar}(s_{\text{current}}$ ,  $H.\text{Evaluate}$ ,  $D$ ,  $N$ );
7  $\mathcal{U}(R, H, D)$ ;
8  $s_{\text{next}} \leftarrow \text{SelectNextState}(R, s_{\text{current}})$ ;
9 if  $\neg \text{Expired}(D)$  then
10  RefineHeuristic( $s_{\text{current}}$ ,  $H$ ,  $D$ );
11 Execute( $s_{\text{next}}$ );
12 RealTimeHeuristicSearch( $s_{\text{next}}$ ,  $H$ ,  $\Delta$ ,  $N$ ,  $\mathcal{U}$ );

```

3.4.1 Heuristic Refinement

Heuristic refinement is independent of the selected real-time search algorithm. Departure-feasibility learning is completed first; spatial-cost learning then uses any remaining time.

Algorithm 2: Incremental heuristic refinement

Input: current state s_{current} , heuristic H , deadline D

```
1 if  $H.$ FinishedLearning( $s_{\text{current}}$ )  $\vee$  Expired( $D$ ) then
2   return;
3 if  $H.$ NeedsDepartureLearning( $s_{\text{current}}$ ) then
4    $H.$ ContinueDepartureLearning( $D$ );
5   if Expired( $D$ ) then
6     return;
7 if  $H.$ NeedsSpatialLearning() then
8    $H.$ ContinueSpatialLearning( $D$ );
```

Algorithm 3: Heuristic evaluation

Input: state s , heuristic H
Output: heuristic value $h(s)$

```
1 if  $H.$ DepartureCost( $s$ ) =  $\infty$  then
2   return  $\infty$ ;
3  $h_{\text{spatial}} \leftarrow H.$ SpatialCost( $s$ );
4 if  $h_{\text{spatial}} = \infty$  then
5   return  $\infty$ ;
6  $h_{\text{adaptive}} \leftarrow H.$ AdaptiveLowerBound( $s$ );
7 return  $\max\{h_{\text{spatial}}, h_{\text{adaptive}}\}$ ;
```

3.4.2 Local Heuristic Updates

RTAA performs a direct linear update to all the closed states.

Algorithm 4: RTAA local heuristic update

Input: lookahead result R , heuristic H , deadline D

```
1  $f_{\text{best}} \leftarrow \text{BestLookaheadValue}(R)$ ;
2 foreach  $s \in R.$ CLOSED do
3   if Expired( $D$ ) then
4     return;
5    $H.$ Update( $s, f_{\text{best}} - R.g(s)$ );
```

LSS-LRTA instead propagates heuristic values backward from OPEN to update the expanded states.

Algorithm 5: LSS-LRTA local heuristic update

Input: lookahead result R , heuristic H , deadline D

```
1  $V \leftarrow \text{BackwardDijkstra}(R.\text{OPEN}, R.\text{CLOSED}, H.\text{Evaluate}, D);$ 
2 if Expired( $D$ ) then
3   return;
4 foreach  $s \in R.\text{CLOSED}$  do
5   if Expired( $D$ ) then
6     return;
7   if  $V(s)$  is defined then
8      $H.\text{Update}(s, V(s));$ 
9   else
10     $H.\text{Update}(s, \infty);$ 
```

4 Completeness & Optimality

The standard guarantees of RTAA and LSS-LRTA do not directly imply completeness in this problem because the robot executes each action immediately and cannot physically back-track. A bounded lookahead search may therefore commit the robot to a state from which interception is no longer possible.

Completeness from the robot's current state at time t is guaranteed only if the departure-feasibility heuristic has been fully learned before the next action is selected. Once learned, this heuristic assigns an infinite value to every state from which the target cannot be intercepted within the remaining time. Consequently, regardless of the expansion limit, neither algorithm will select an infeasible successor while a feasible successor exists. If every available successor has an infinite heuristic value, then no feasible interception path remains. Otherwise, RTAA and LSS-LRTA select a finite- f successor, which is guaranteed to preserve feasibility.

Without the departure heuristic, neither algorithm is complete under bounded lookahead: the robot may enter a state that appears promising locally but leads to an unavoidable dead end. Completeness would then require an expansion limit large enough to search the entire relevant state-time space, or at least enough to establish a feasible continuation before committing to an action. Moreover, bounded real-time search does not generally guarantee a globally optimal trajectory; larger expansion limits can improve solution quality but do not guarantee optimality unless the complete relevant search space is considered.

5 Results

The results of running the two algorithms under different expansion limits are shown below.

Map 1

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	5316
Total planning time (s)	10.038632
Moves made	5027
Path cost	5316

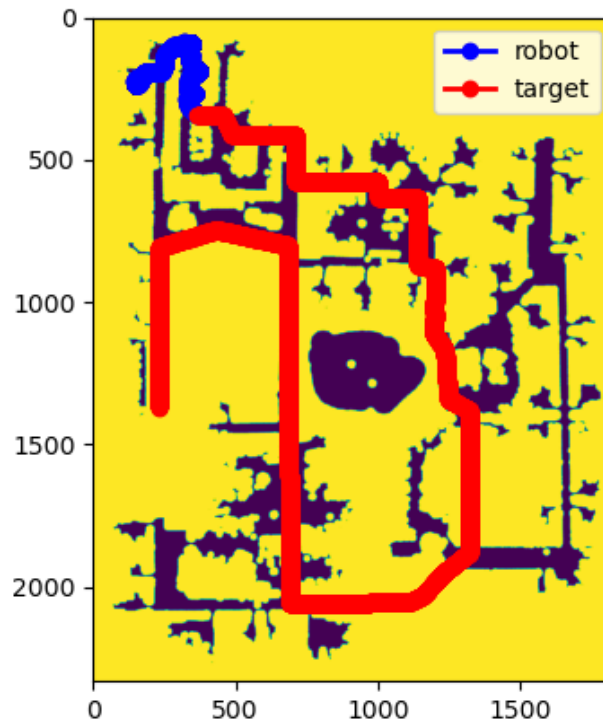


Figure 1: Final robot and target trajectories for Map 1 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	5316
Total planning time (s)	13.109973
Moves made	5314
Path cost	5316

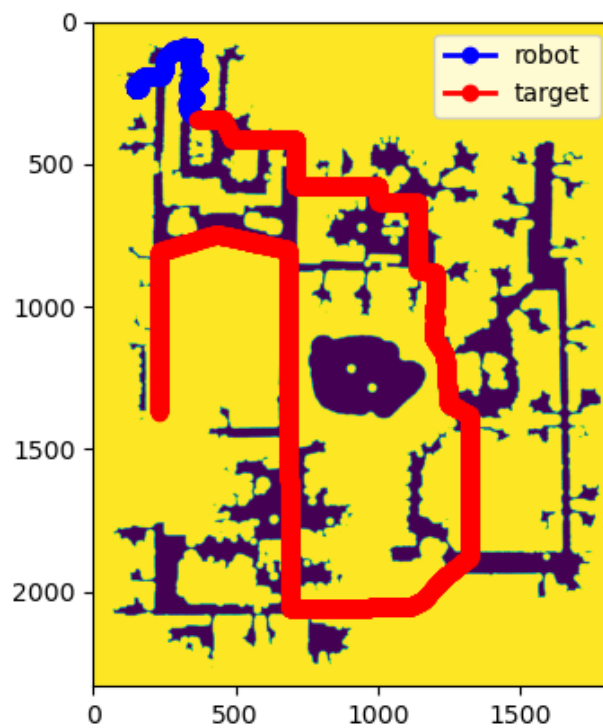


Figure 2: Final robot and target trajectories for Map 1 using LSS-LRTA with expansion limit 100.

Map 2

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	5217
Total planning time (s)	17.070871
Moves made	5216
Path cost	2852354

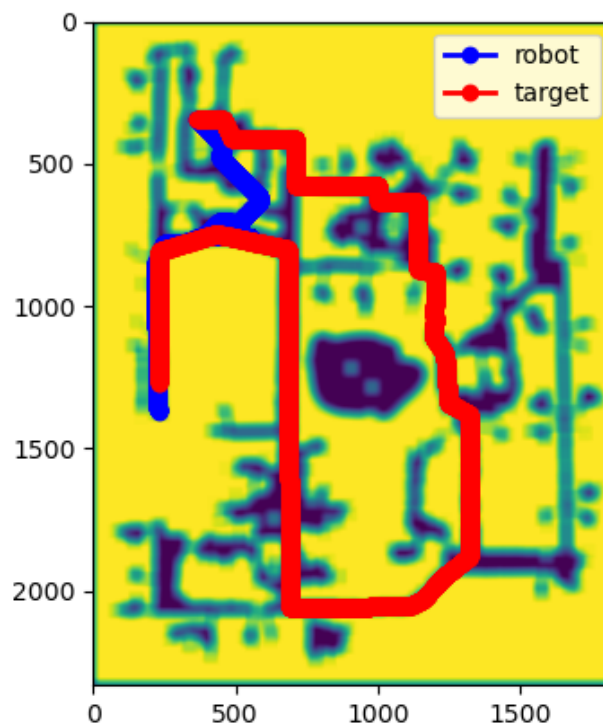


Figure 3: Final robot and target trajectories for Map 2 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	5217
Total planning time (s)	21.782582
Moves made	5211
Path cost	2851461

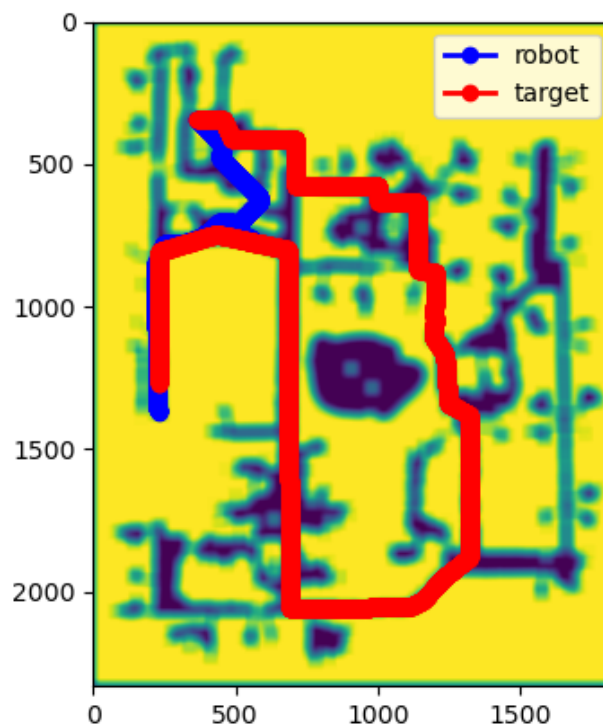


Figure 4: Final robot and target trajectories for Map 2 using LSS-LRTA with expansion limit 100.

Map 3

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	241
Total planning time (s)	1.141930
Moves made	241
Path cost	241

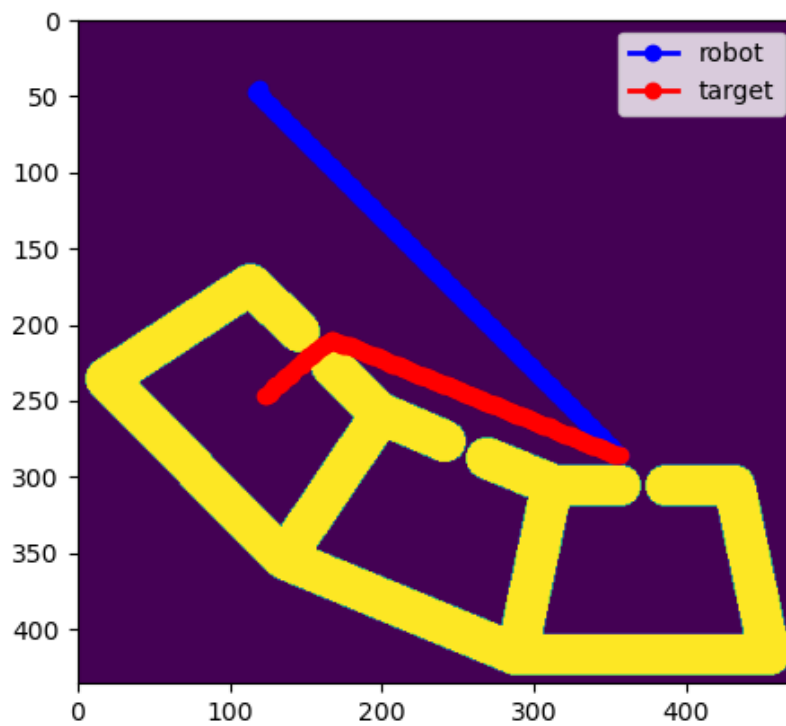


Figure 5: Final robot and target trajectories for Map 3 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	241
Total planning time (s)	1.707082
Moves made	241
Path cost	241

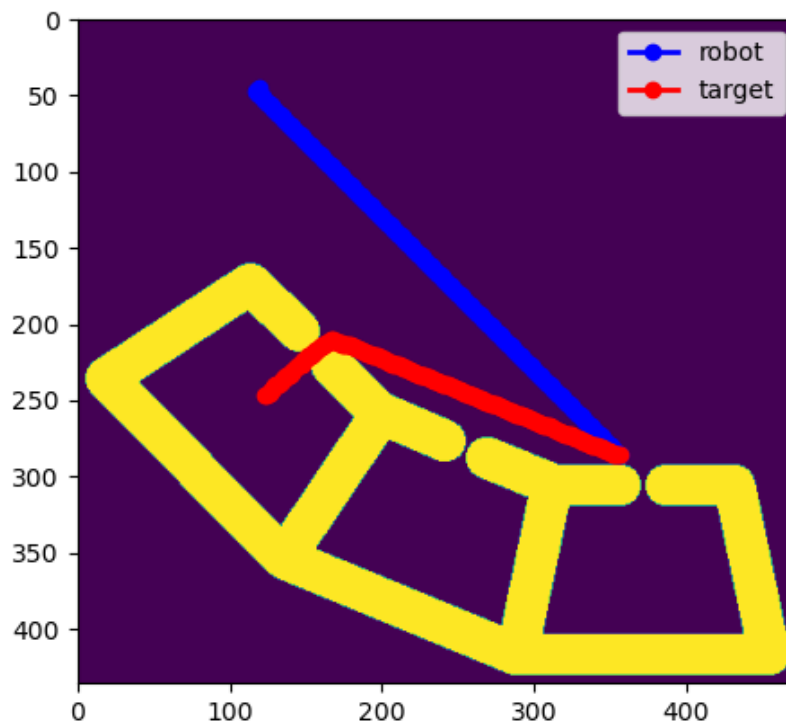


Figure 6: Final robot and target trajectories for Map 3 using LSS-LRTA with expansion limit 100.

Map 4

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	377
Total planning time (s)	1.334541
Moves made	377
Path cost	377

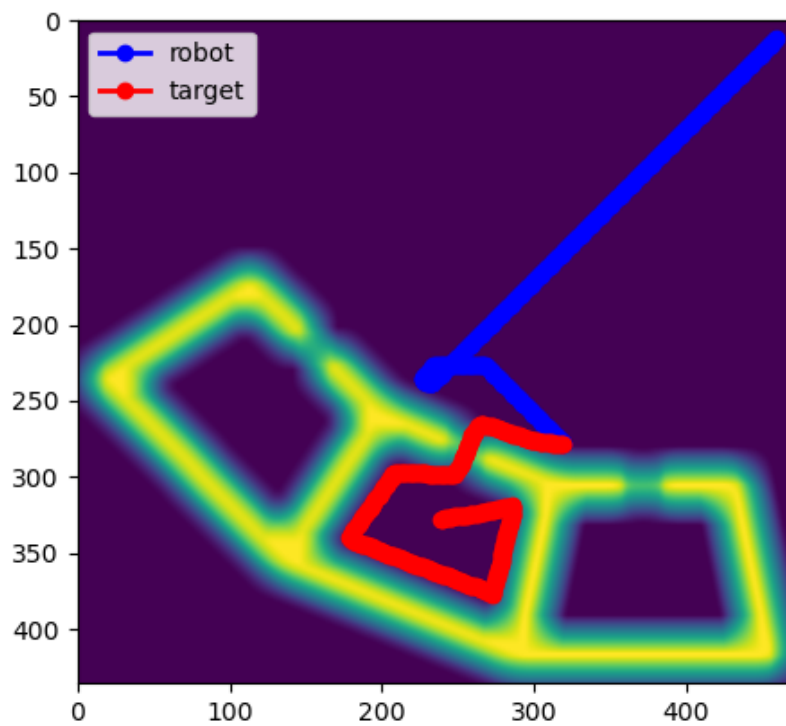


Figure 7: Final robot and target trajectories for Map 4 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	377
Total planning time (s)	1.700465
Moves made	377
Path cost	377

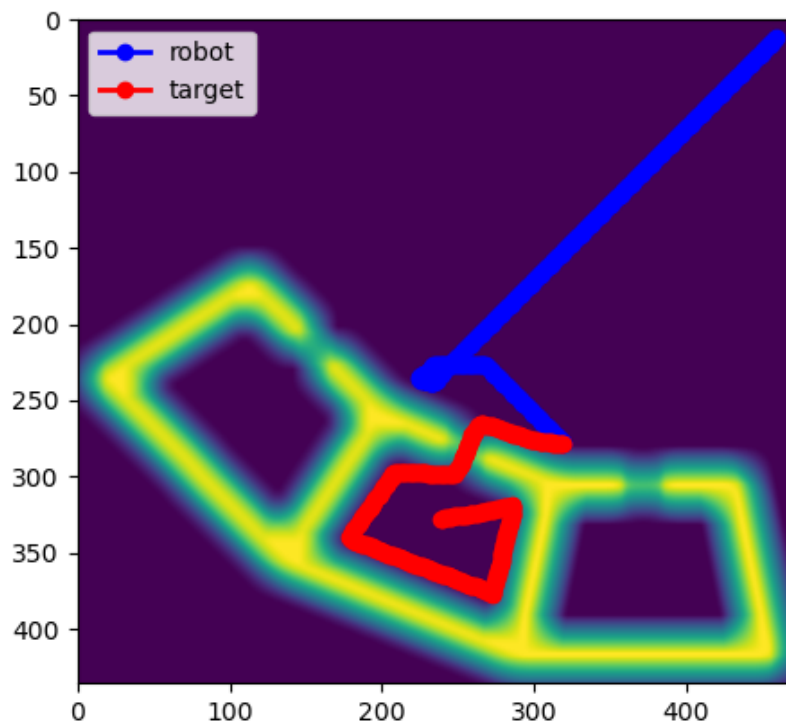


Figure 8: Final robot and target trajectories for Map 4 using LSS-LRTA with expansion limit 100.

Map 5

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	181
Total planning time (s)	0.314166
Moves made	181
Path cost	1357

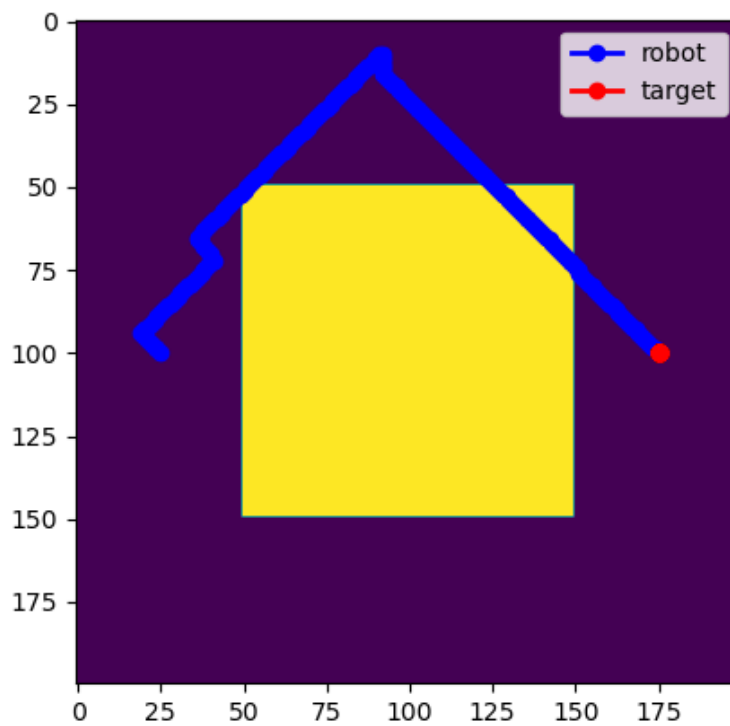


Figure 9: Final robot and target trajectories for Map 5 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	181
Total planning time (s)	0.520379
Moves made	181
Path cost	1357

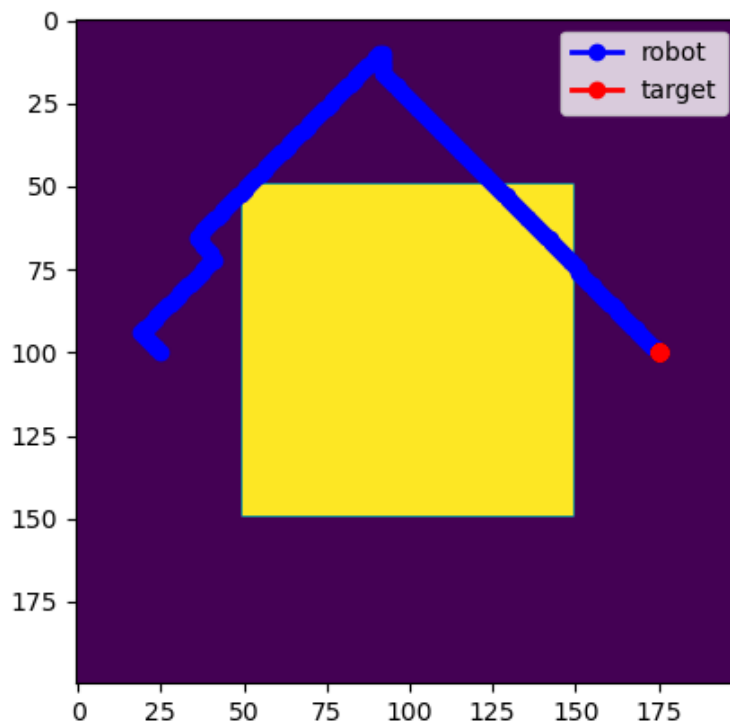


Figure 10: Final robot and target trajectories for Map 5 using LSS-LRTA with expansion limit 100.

RTAA 500

Metric	Value
Target caught	Yes
Time taken (s)	181
Total planning time (s)	0.718257
Moves made	181
Path cost	1210

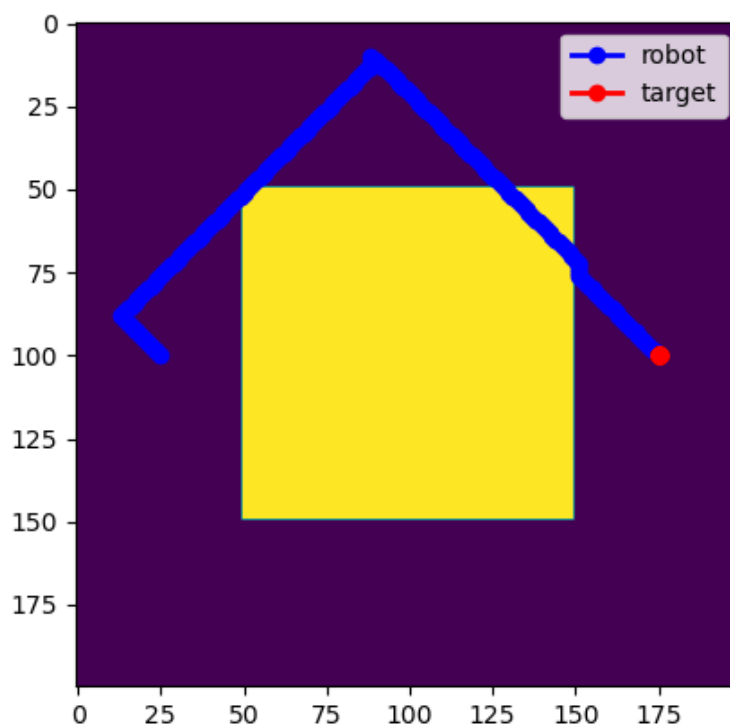


Figure 11: Final robot and target trajectories for Map 5 using RTAA with expansion limit 500.

LSS-LRTA 500

Metric	Value
Target caught	Yes
Time taken (s)	181
Total planning time (s)	1.285368
Moves made	181
Path cost	1259

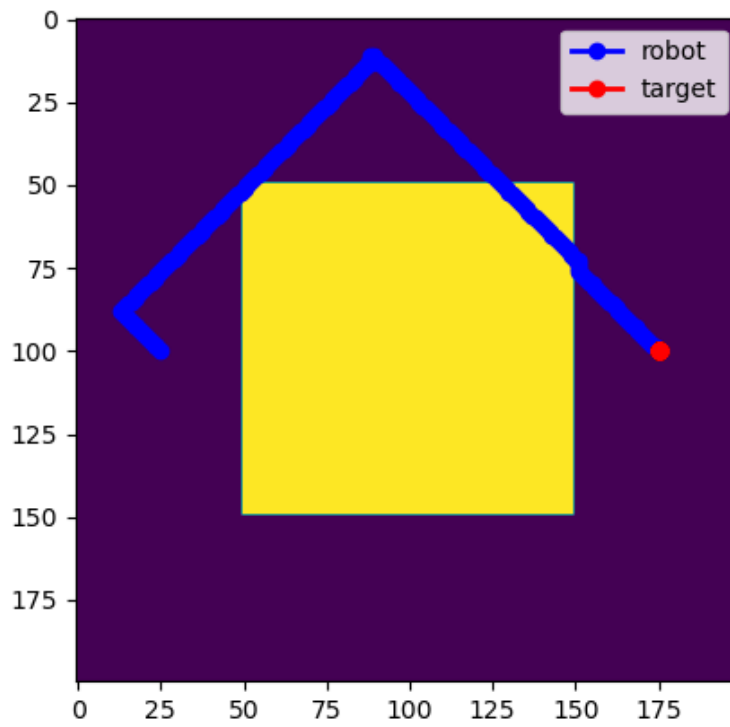


Figure 12: Final robot and target trajectories for Map 5 using LSS-LRTA with expansion limit 500.

Map 6

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	125
Total planning time (s)	0.283464
Moves made	125
Path cost	2500

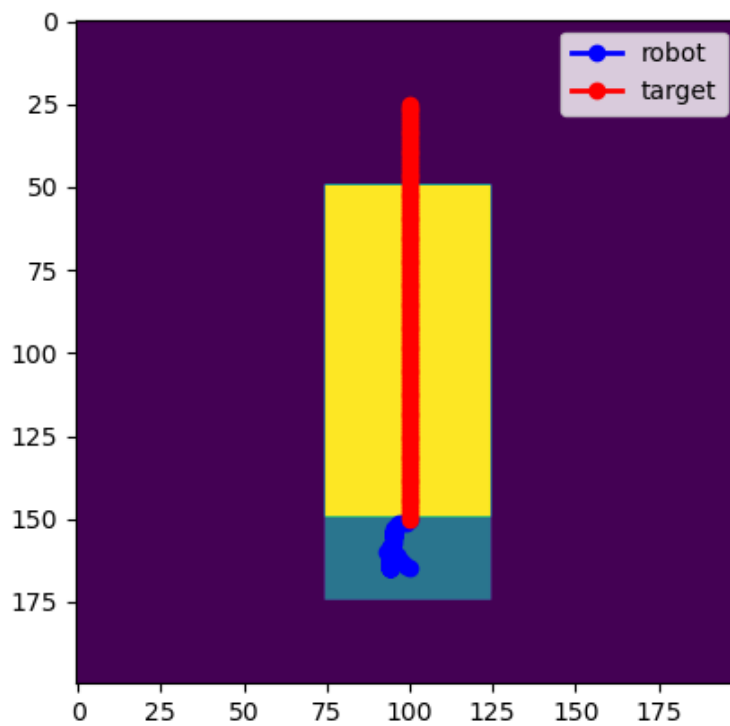


Figure 13: Final robot and target trajectories for Map 6 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	125
Total planning time (s)	0.372342
Moves made	125
Path cost	2500

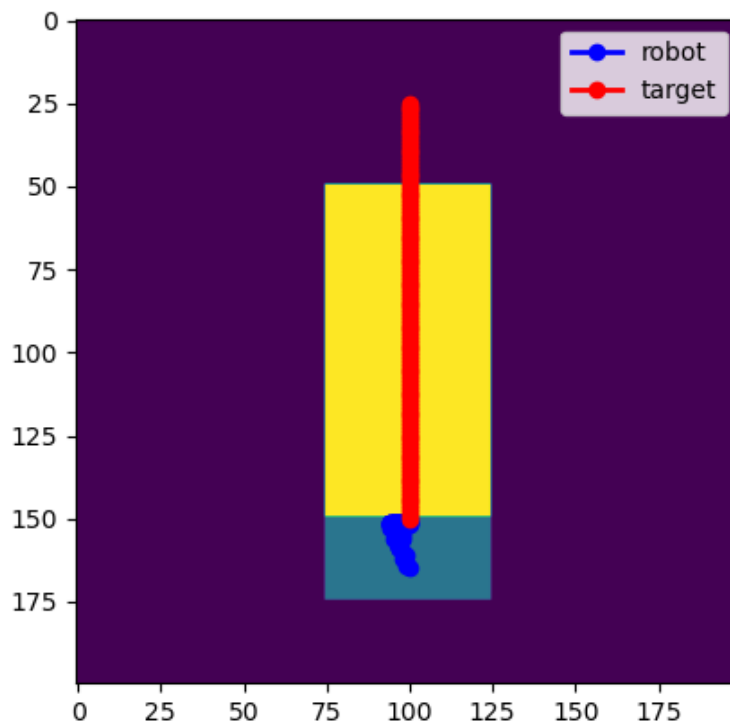


Figure 14: Final robot and target trajectories for Map 6 using LSS-LRTA with expansion limit 100.

RTAA 1000

Metric	Value
Target caught	Yes
Time taken (s)	140
Total planning time (s)	0.910492
Moves made	140
Path cost	843

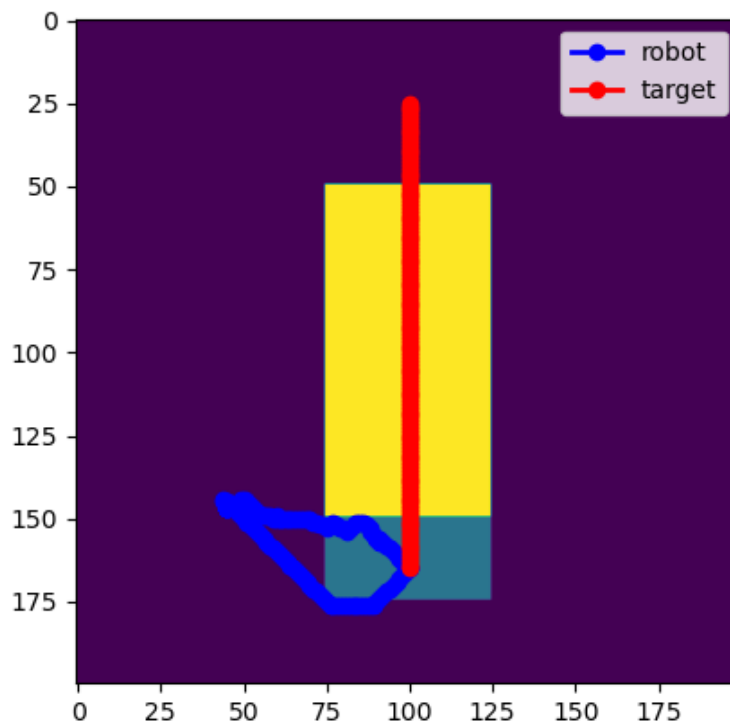


Figure 15: Final robot and target trajectories for Map 6 using RTAA with expansion limit 1000.

LSS-LRTA 1000

Metric	Value
Target caught	Yes
Time taken (s)	140
Total planning time (s)	1.466185
Moves made	140
Path cost	1014

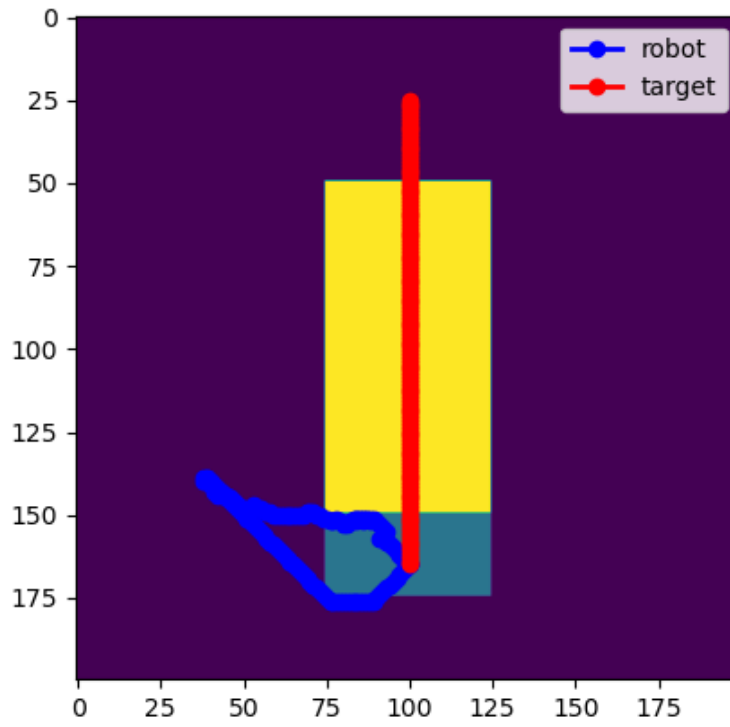


Figure 16: Final robot and target trajectories for Map 6 using LSS-LRTA with expansion limit 1000.

Map 7

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	250
Total planning time (s)	0.966563
Moves made	250
Path cost	250

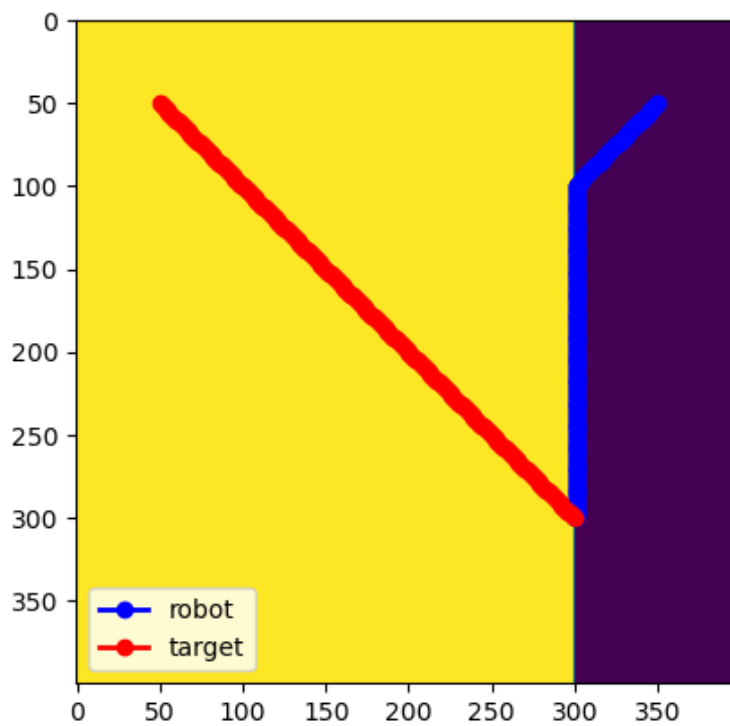


Figure 17: Final robot and target trajectories for Map 7 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	250
Total planning time (s)	1.204509
Moves made	250
Path cost	250

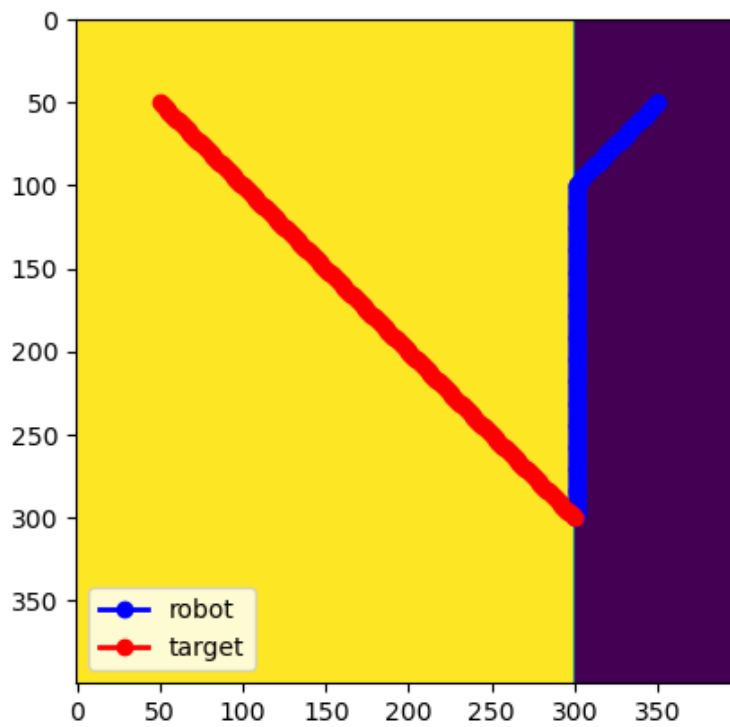


Figure 18: Final robot and target trajectories for Map 7 using LSS-LRTA with expansion limit 100.

Map 8

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	450
Total planning time (s)	0.901593
Moves made	450
Path cost	450

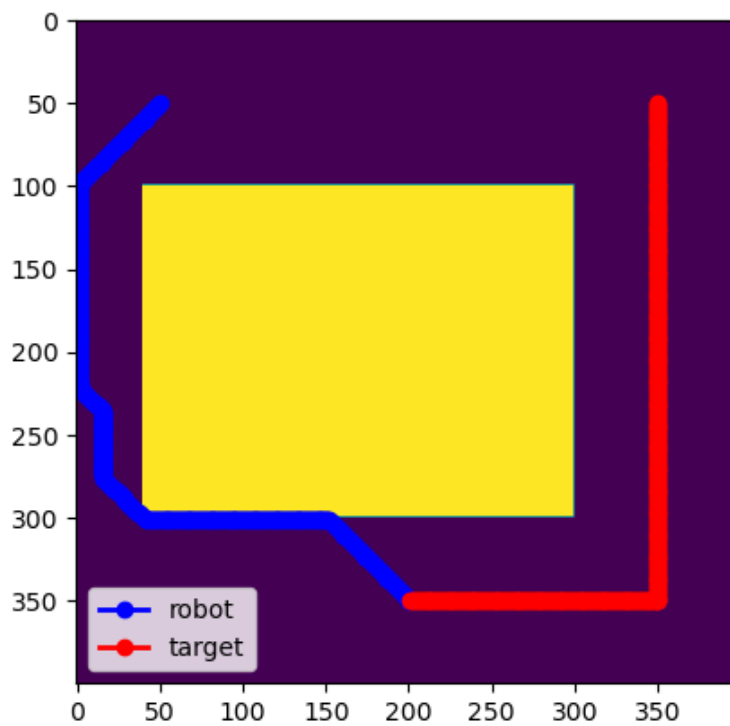


Figure 19: Final robot and target trajectories for Map 8 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	450
Total planning time (s)	1.369250
Moves made	450
Path cost	450

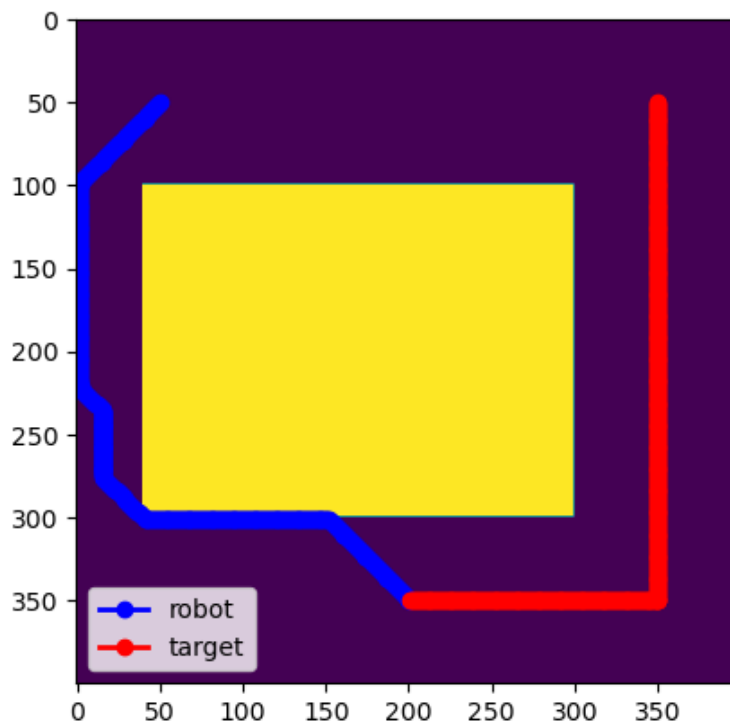


Figure 20: Final robot and target trajectories for Map 8 using LSS-LRTA with expansion limit 100.

Map 9

RTAA 100

Metric	Value
Target caught	Yes
Time taken (s)	400
Total planning time (s)	1.130461
Moves made	400
Path cost	400

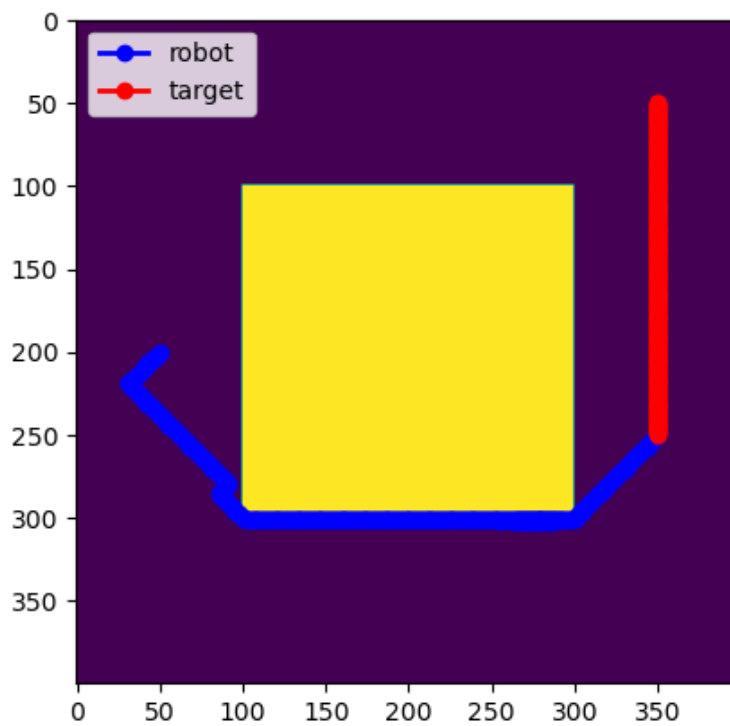


Figure 21: Final robot and target trajectories for Map 9 using RTAA with expansion limit 100.

LSS-LRTA 100

Metric	Value
Target caught	Yes
Time taken (s)	400
Total planning time (s)	2.374034
Moves made	400
Path cost	400

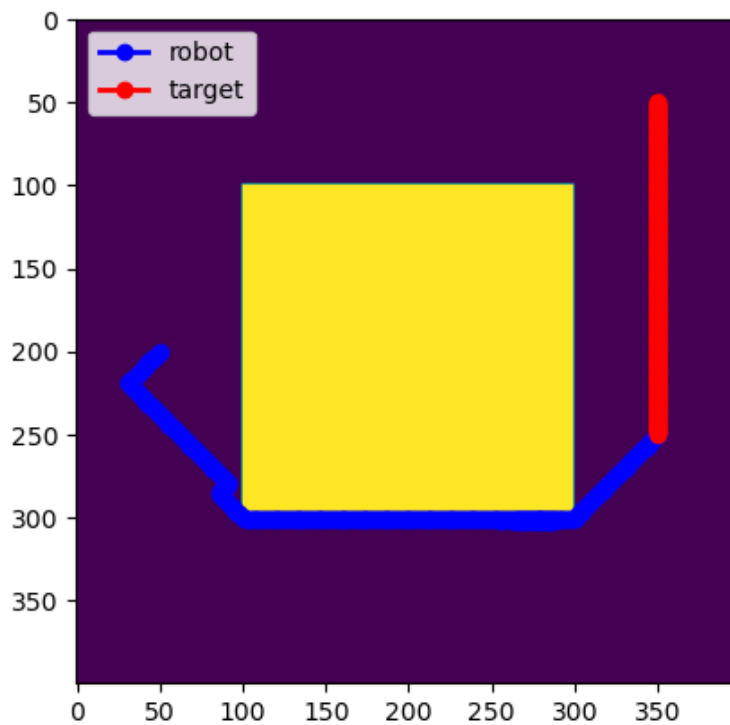


Figure 22: Final robot and target trajectories for Map 9 using LSS-LRTA with expansion limit 100.

References

- [1] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.
- [2] Sven Koenig and Xiaoxun Sun. Comparing real-time and incremental heuristic search for real-time situated agents. *Autonomous Agents and Multi-Agent Systems*, 18(3):313–341, 2009.
- [3] Richard E. Korf. Real-time heuristic search. *Artificial Intelligence*, 42(2–3):189–211, 1990.
- [4] Sven Koenig and Maxim Likhachev. Real-time adaptive A*. In *Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems*, pages 281–288. ACM, 2006.